Introduction
Part 2
Data Structure Algorithm

Ashim Khadka

Master Program

Outline

1. Divide and Conquer
2. Greedy Algorithms
3. Dynamic Programming

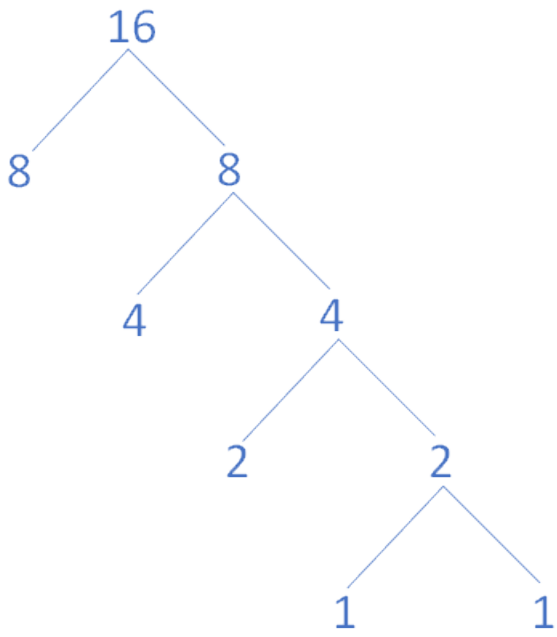
Detecting Counterfeit Coin

Given a bag containing 16 coins, and told that one of these coins may be counterfeit and lighter than the genuine ones. Determine whether the bag contains a counterfeit coin.

- Compare the weights of coins 1 and 2
  - If coin 1 is lighter than coin 2, then coin 1 is counterfeit
  - If coin 2 is lighter than coin 1, then coin 2 is counterfeit
  - If both coins have the same weight, compare coin 3 and 4, and so on
  - 8 weight comparisons need

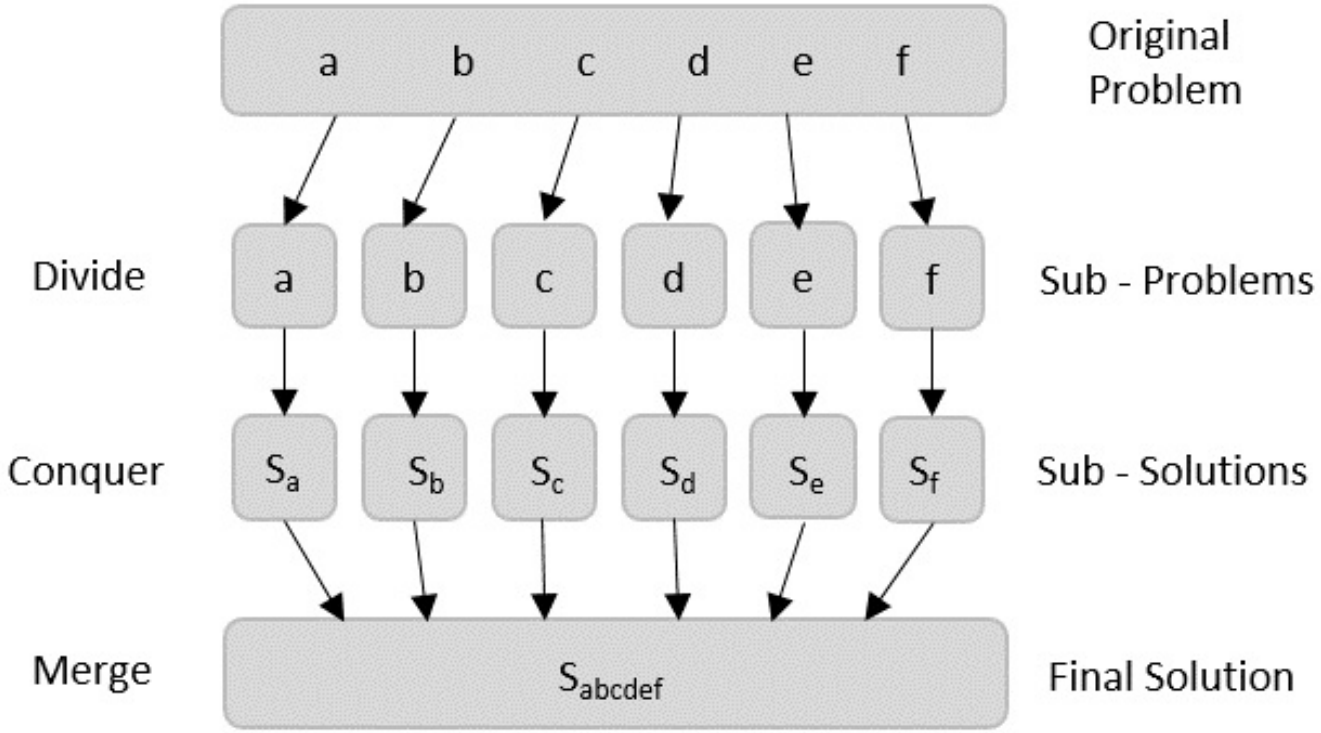
Detecting Counterfeit Coin

1. Divide into 2 groups of 8 coins each
   - Weigh 8 coins against another 8 coins
   - If they balance, there is no counterfeit
   - If not, the lighter side contains the counterfeit
2. Divide the suspect group of 8 into 2 groups of 4 coins each
   - Weigh 4 coins against another 4 coins
   - Same logic: if balanced, no counterfeit; if not, lighter side has it
3. Divide the suspect group of 4 into 2 groups of 2 coins each
   - Weigh 2 coins against another 2 coins
   - Same logic: if balanced, no counterfeit; if not, lighter side has it
4. Divide the suspect group of 2 into 2 groups of 1 coins each
   - Weigh 1 coins against another 1 coins
   - Same logic: if balanced, no counterfeit; if not, lighter side has it



Divide and Conquer

- solves complex problems by recursively breaking them down into two or more smaller
- manageable sub-problems of the same type until they are simple enough to be solved directly
- sub-problem solutions are then combined to solve the original problem



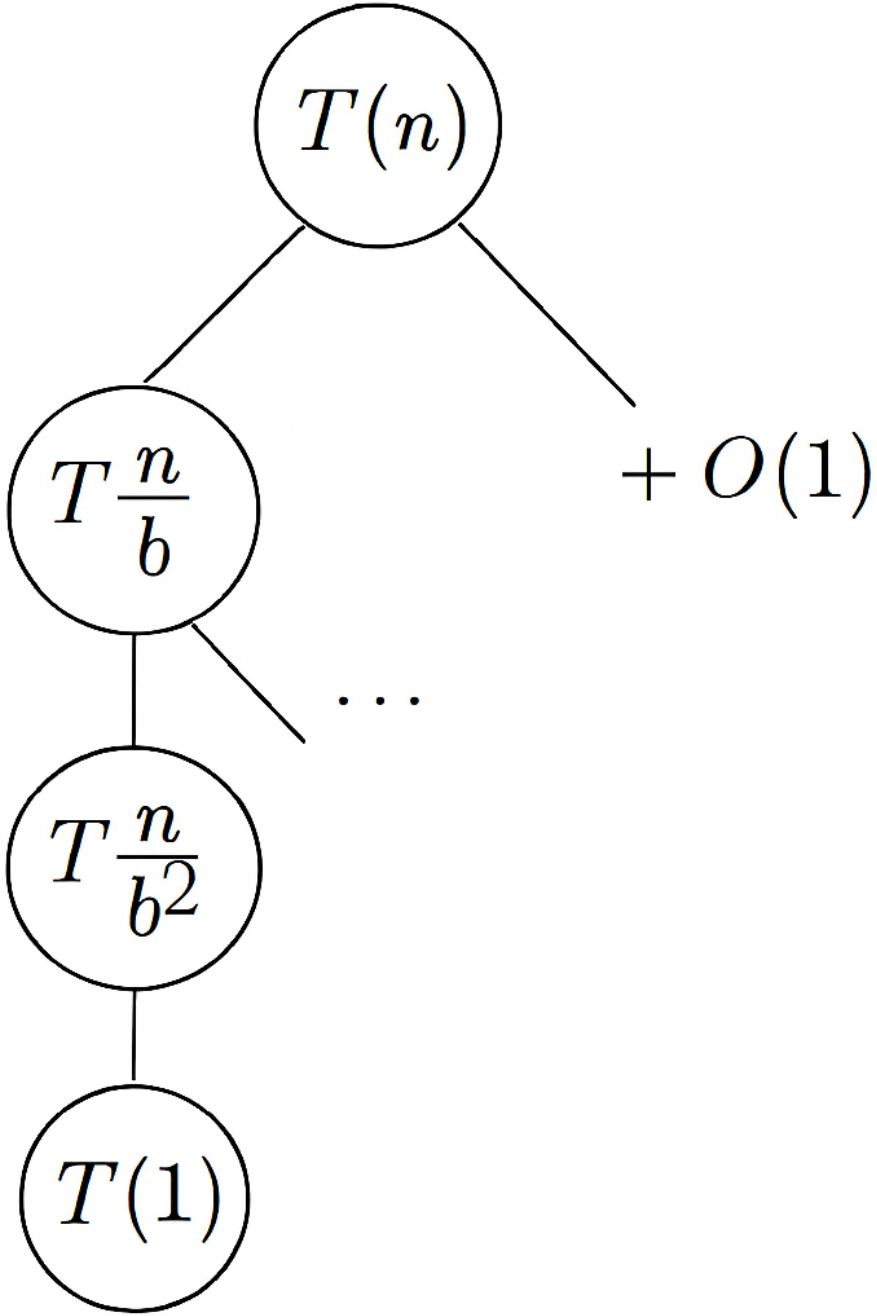
- Let $T(n)$ denote the worst-case time this algorithm takes to solve a problem of size n.
- T (n) is given by the following recurrence relation:

$$T(n) = aT(n/b) + f(n)$$

- a: The number of subproblems created in each recursive step ($a \geq 1$)
- n/b: The size of each subproblem ($b > 1$)
- $f(n)$: The "extra work" performed outside the recursive calls, typically the cost of the divide and combine phases.

Common Approach

1. Max-Min Problem
2. Merge Sort Algorithm
3. Quick Sort Algorithm
4. Binary Search Algorithm
5. Strassen's Matrix Multiplication
6. Karatsuba Algorithm
7. Convex Hull


$T(n)$
$T\frac{n}{b}$
$+ O(1)$
$\cdots$
$T\frac{n}{b^2}$
$T(1)$

Recall: Detecting Counterfeit Coin Complexity

$$T(n) = \begin{cases} T(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

- $a = 1$: only one subproblem continues
- $b = 2$: the problem size is halved each time
- $f(n) = O(1)$: weighing costs constant time

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + 1 \\ &= T\left(\frac{n}{2^2}\right) + 2 \\ &= T\left(\frac{n}{2^3}\right) + 3 \\ &\vdots \\ &= T\left(\frac{n}{2^k}\right) + k \end{aligned}$$

$$\begin{aligned} \frac{n}{b^k} &= 1 \\ n &= b^k \\ k &= \log_b n \end{aligned}$$

- $T(n) = T(1) + \log_b n$
- $T(n) = O(\log_b n)$

Max-Min Problem

- find the maximum and minimum value in a given array number[] of size n
- **Naive Method:** the maximum and minimum element found by comparing each element and updating Max and Min values as and when required
- **approach:** simple, $(n-1)$ comparisons for finding max and the same number of comparisons for finding the min
- results in a total of $2(n-1)$ comparisons

Divide and Conquer

- array: divides into two halves
- using a recursive approach maximum and minimum numbers in each halves are found
- return the maximum of two maxima of each half and the minimum of two minima of each half

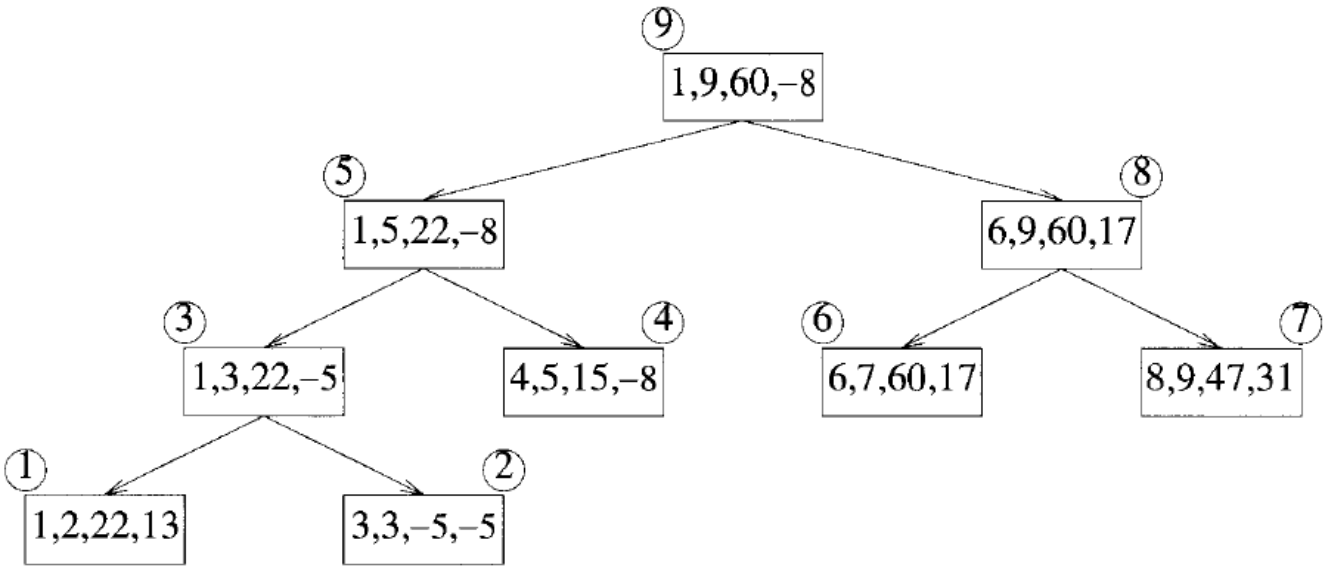
Max-Min Problem: Algorithm

Each node has four items of information: i, j, max, min

```
Algorithm MaxMin(i, j, max, min)
// a[1 : n] is a global array. Parameters i and j are integers,
// 1 ≤ i ≤ j ≤ n. The effect is to set max and min to the
// largest and smallest values in a[i : j], respectively.
{
    if (i = j) then max := min := a[i]; // Small(P)
    else if (i = j − 1) then  // Another case of Small(P)
        {
            if (a[i] < a[j]) then
            {
                max := a[j]; min := a[i];
            }
            else
            {
                max := a[i]; min := a[j];
            }
        }
        else
        {   // If P is not small, divide P into subproblems.
            // Find where to split the set.
                mid := ⌊(i + j)/2⌋;
            // Solve the subproblems.
                MaxMin(i, mid, max, min);
                MaxMin(mid + 1, j, max1, min1);
            // Combine the solutions.
                if (max < max1) then max := max1;
                if (min > min1) then min := min1;
        }
}
```

Example

a:	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
	22	13	-5	-8	15	60	17	31	47



Max-Min Problem: Complexity

MaxMin recurrence relation

$$T(n) = \begin{cases} 0 & n = 1 \\ 1 & n = 2 \\ T(n/2) + T(n/2) + 2 & n > 2 \end{cases}$$

When n is a power of two, $n = 2^k$, then

$$\begin{aligned} T(n) &= 2T(n/2) + 2 \\ &= 2(2T(n/2) + 2) + 2 \\ &= 4T(n/4) + 4 + 2 \\ &\vdots \\ &= 2^k T(n/2^k) + (2^k - 1) \cdot 2 \end{aligned}$$

$$\begin{aligned} \frac{n}{2^k} &= 2 \\ 2^k &= n/2 \\ T(n) &= \frac{n}{2} T(2) + \left(\frac{n}{2} - 1\right) 2 \\ &= \frac{3n}{2} - 2 \end{aligned}$$

Assignment

Divide-and-Conquer Min-Max vs. Tournament Tree:
Implement both approaches, prove the comparison bounds, and experimentally validate with large arrays

Strassen Matrix Multiplication

- divide and conquer approach to solve the matrix multiplication problems
- usual matrix multiplication method multiplies each row with each column to achieve the product matrix
  - time complexity: $O(n^3)$
- Strassens Matrix multiplication: only on square matrices where n is a power of 2
- divide the matrices into four $\frac{n}{2} \times \frac{n}{2}$ blocks:

$$Z = \begin{bmatrix} I & K \\ J & L \end{bmatrix}, \quad X = \begin{bmatrix} A & C \\ B & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & G \\ F & H \end{bmatrix}$$

- Using Strassen's algorithm

$$\begin{aligned}
M_1 &= (A + C)(E + F), \quad M_2 = (B + D)(G + H), \\
M_3 &= (A - D)(E + H), \quad M_4 = A(F - H), \\
M_5 &= (C + D)E, \quad M_6 = (A + B)H, \\
M_7 &= D(G - E).
\end{aligned}$$

Then the resulting blocks are:

$$
\begin{aligned}
I &= M_2 + M_3 - M_6 - M_7, \\
J &= M_4 + M_6, \\
K &= M_5 + M_7, \\
L &= M_1 - M_3 - M_4 - M_5.
\end{aligned}
$$

Recurrence Analysis: The recurrence relation for Strassen's algorithm is:

$$
T(n) = \begin{cases} c & \text{if } n = 1, \\ 7 \cdot T\left(\frac{n}{2}\right) + d \cdot n^2 & \text{otherwise,} \end{cases}
$$

where c and d are constants.
By the Master Theorem, we compare $n^{\log_2 7}$ with n^2:

$$
\log_2 7 \approx 2.807 > 2,
$$

so the recurrence solves to:

$$
T(n) = \Theta\left(n^{\log_2 7}\right)
$$

Thus,

$$
T(n) = O\left(n^{\log_2 7}\right).
$$

Karatsuba Algorithm

- a fast, divide-and-conquer algorithm for multiplying large integers, discovered by Anatoly Karatsuba in 1960
  - **Split:** n-digit numbers into smaller parts ($n/2$ digits) to reduce computational effort
    - Divide X and Y into high and low parts: $X = a \cdot 10^{\lfloor n/2 \rfloor} + b$, $Y = c \cdot 10^{\lfloor n/2 \rfloor} + d$
  - **Multiply (Recursive Steps):**
    - $z_0 = a \cdot c$ (Product of high parts)
    - $z_2 = b \cdot d$ (Product of low parts)
    - $z_1 = (a + b) \cdot (c + d) - z_0 - z_2$ (Middle terms)
  - Combine the results: $Result = z_0 \cdot 10^{2\lfloor n/2 \rfloor} + z_1 \cdot 10^{\lfloor n/2 \rfloor} + z_2$
- reduces multiplication complexity from $O(n^2)$ (grade-school method) to approximately $O(n^{\log_2 3}) \approx O(n^{1.585})$ by reducing four recursive multiplications to three

Karatsuba Multiplication: Recursive Breakdown

Let:

$$X = 146123 = a \cdot 10^3 + b, \quad a = 146,\ b = 123$$

$$Y = 352120 = c \cdot 10^3 + d, \quad c = 352,\ d = 120$$

Karatsuba requires:

- $ac = 146 \times 352$
- $bd = 123 \times 120$
- $(a + b)(c + d) = (146 + 123)(352 + 120)$

Karatsuba Multiplication: Recursive Breakdown

Let:

$$X = 146123 = a \cdot 10^3 + b, \quad a = 146,\ b = 123$$

$$Y = 352120 = c \cdot 10^3 + d, \quad c = 352,\ d = 120$$

Karatsuba requires:

- $ac = 146 \times 352$
- $bd = 123 \times 120$
- $(a + b)(c + d) = (146 + 123)(352 + 120)$

Recursive Karatsuba on 146×352:

$$146 = a_1 \cdot 10 + b_1 = 14 \cdot 10 + 6, \quad 352 = c_1 \cdot 10 + d_1 = 35 \cdot 10 + 2$$

- $a_1 c_1 = 14 \times 35$
- $b_1 d_1 = 6 \times 2 = 12$
- $(a_1 + b_1)(c_1 + d_1) = 20 \times 37$
- Cross term: $ad + bc = (a_1 + b_1)(c_1 + d_1) - a_1 c_1 - b_1 d_1$

Karatsuba Multiplication: Recursive Breakdown

Recursive Karatsuba on 14×35:

$$14 = a_{11} \cdot 10 + b_{11} = 1 \cdot 10 + 4, \quad 35 = c_{11} \cdot 10 + d_{11} = 3 \cdot 10 + 5$$

- $a_{11}c_{11} = 1 \times 3 = 3$
- $b_{11}d_{11} = 4 \times 5 = 20$
- $(a_{11} + b_{11})(c_{11} + d_{11}) = 5 \times 8 = 40$
- Cross term:
  $a_{11}d_{11} + b_{11}c_{11} = (a_{11} + b_{11})(c_{11} + d_{11}) - a_{11}c_{11} - b_{11}d_{11} = 40 - 20 - 3 = 17$

Karatsuba Multiplication: Recursive Breakdown

Recursive Karatsuba on 14×35:

$$14 = a_{11} \cdot 10 + b_{11} = 1 \cdot 10 + 4, \quad 35 = c_{11} \cdot 10 + d_{11} = 3 \cdot 10 + 5$$

- $a_{11}c_{11} = 1 \times 3 = 3$
- $b_{11}d_{11} = 4 \times 5 = 20$
- $(a_{11} + b_{11})(c_{11} + d_{11}) = 5 \times 8 = 40$
- Cross term:
  $a_{11}d_{11} + b_{11}c_{11} = (a_{11} + b_{11})(c_{11} + d_{11}) - a_{11}c_{11} - b_{11}d_{11} = 40 - 20 - 3 = 17$

Recombine:

$$14 \times 35 = 3 \cdot 10^2 + 17 \cdot 10 + 20 = 490$$

Karatsuba Multiplication: Recursive Breakdown

Recursive Karatsuba on 14×35:

$$14 = a_{11} \cdot 10 + b_{11} = 1 \cdot 10 + 4, \quad 35 = c_{11} \cdot 10 + d_{11} = 3 \cdot 10 + 5$$

- $a_{11}c_{11} = 1 \times 3 = 3$
- $b_{11}d_{11} = 4 \times 5 = 20$
- $(a_{11} + b_{11})(c_{11} + d_{11}) = 5 \times 8 = 40$
- Cross term:
  $a_{11}d_{11} + b_{11}c_{11} = (a_{11} + b_{11})(c_{11} + d_{11}) - a_{11}c_{11} - b_{11}d_{11} = 40 - 20 - 3 = 17$

Recombine:

$$14 \times 35 = 3 \cdot 10^2 + 17 \cdot 10 + 20 = 490$$

Similarly: $(a_1 + b_1)(c_1 + d_1) = 20 \times 37$
Recursive Karatsuba on 20×37:

$$20 \times 37 = 6 \cdot 10^2 + 14 \cdot 10 + 0 = 740$$

Karatsuba Multiplication: Recursive Breakdown

- $a_1 c_1 = 14 \times 35 = 490$
- $b_1 d_1 = 6 \times 2 = 12$
- $(a_1 + b_1)(c_1 + d_1) = 20 \times 37 = 740$
- Cross term: $a_1 d_1 + b_1 c_1 = 740 - 490 - 12 = 238$

Karatsuba Multiplication: Recursive Breakdown

- $a_1 c_1 = 14 \times 35 = 490$
- $b_1 d_1 = 6 \times 2 = 12$
- $(a_1 + b_1)(c_1 + d_1) = 20 \times 37 = 740$
- Cross term: $a_1 d_1 + b_1 c_1 = 740 - 490 - 12 = 238$

Recombine:

$$146 \times 352 = 490 \cdot 10^2 + 238 \cdot 10^1 + 12 = 51392$$

Karatsuba Multiplication: Recursive Breakdown

- $a_1 c_1 = 14 \times 35 = 490$
- $b_1 d_1 = 6 \times 2 = 12$
- $(a_1 + b_1)(c_1 + d_1) = 20 \times 37 = 740$
- Cross term: $a_1 d_1 + b_1 c_1 = 740 - 490 - 12 = 238$

Recombine:

$$146 \times 352 = 490 \cdot 10^2 + 238 \cdot 10^1 + 12 = 51392$$

Similarly:

- $ac = 146 \times 352 = 51392$
- $bd = 123 \times 120 = 14760$
- $(a + b)(c + d) = (146 + 123)(352 + 120) = 126968$
- Cross term: $ad + bc = 126968 - 51392 - 14760 = 60816$

Karatsuba Multiplication: Recursive Breakdown

- $a_1 c_1 = 14 \times 35 = 490$
- $b_1 d_1 = 6 \times 2 = 12$
- $(a_1 + b_1)(c_1 + d_1) = 20 \times 37 = 740$
- Cross term: $a_1 d_1 + b_1 c_1 = 740 - 490 - 12 = 238$

Recombine:

$$146 \times 352 = 490 \cdot 10^2 + 238 \cdot 10^1 + 12 = 51392$$

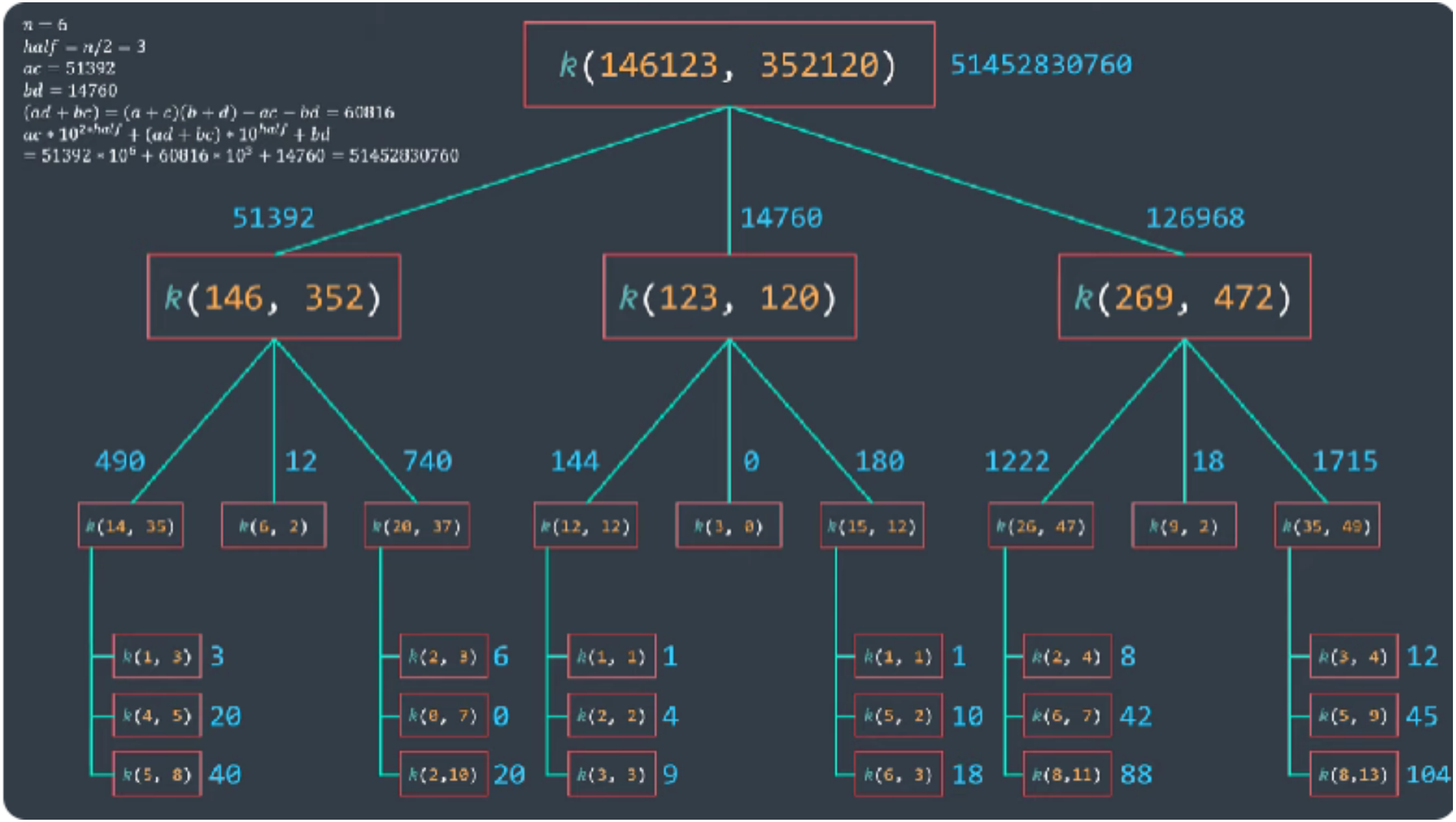
Similarly:

- $ac = 146 \times 352 = 51392$
- $bd = 123 \times 120 = 14760$
- $(a + b)(c + d) = (146 + 123)(352 + 120) = 126968$
- Cross term: $ad + bc = 126968 - 51392 - 14760 = 60816$

Final recombination:

$$X \cdot Y = 51392 \cdot 10^6 + 60816 \cdot 10^3 + 14760 = 51452830760$$

Karatsuba Algorithm



Assignment

Convex Hull

- QuickHull Algorithm
- Graham's Scan
- Divide-and-Conquer Algorthim

Greedy Algorithms

- algorithmic strategy that makes the best immediate, locally optimal choice at each step
  - hope that these choices will lead to a globally optimal solution
- follows a straightforward strategy: at every stage
  - picks the option that looks most favorable right now without considering the long-term impact of that decision
- do not revisit past decisions or adjust their approach based on future possibilities

Minimum number of coins

- N= 143 given that 1000, 500, 100, 50, 20, 10, 5, 2, 1
- N = 100+20+20+2+1
  - 5 notes selected
- N = 6, given that 4,3,1
- N = 4+1+1 Suboptimal
  - Optimal N= 3+3

Shop in Candy Store

Given an array prices[] of n different candies, where prices[i] represents the price of the i-th candy and all prices are distinct. The store has an offer where for every candy you buy, you can get up to k other different candies for free. Find the minimum and maximum amount of money needed to buy all the candies.

5 3 2 1 4

- N=5, k=2

idea:

- to minimize the cost by buying the cheapest candies and taking the most expensive ones for free
- to maximize the cost by buying the most expensive candies and taking the cheapest ones for free

Sort the prices array in ascending order to access the cheapest and most expensive candies easily

For minimum cost

- use a pointer i starting from the beginning and a variable n for the total candies
- each step: buy the candy at i (add to minCost) and move i forward
- reduce n by k to simulate getting up to k candies for free

For maximum cost

- use a pointer i starting from the end and an index marker initialized to -1
- each step: buy the candy at i (add to maxCost) and move i backward
- increase index by k to simulate taking free candies

Return both minCost and maxCost as the final result

Dynamic Programming

- algorithmic optimization technique for solving complex problems by breaking them into
  - simpler, overlapping subproblems and storing their solutions to avoid recomputation
- transforms inefficient recursive solutions with exponential time complexities into more efficient polynomial-time algorithms
  - using data structures like arrays or hash tables to store intermediate results
- optimization problem: find a solution with the optimal value
  - minimum or maximum

Characteristics

1. Overlapping Subproblems:
   - same subproblems are encountered and solved multiple times during the computation
     - e.g., calculating Fibonacci numbers
2. Optimal Substructure:
   - optimal solution to the overall problem can be constructed from the optimal solutions of its smaller subproblems
     - e.g., the shortest path between two points in a graph contains the shortest paths to all intermediate points

Sequence of four steps

1. Characterize the structure of an optimal solution
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution in a bottom-up fashion
4. Construct an optimal solution from computed information